Shailendra

https://saiinfosoft.co.in/

Sudarshan Nano AI
SaiInfosoft
AI on the Edge

SDN-AI: Beyond the Transformer Paradigm

A Dynamic Equation System for Ultra-Lightweight Edge Automation

Version 1.0 – May 2026
Author: Shailendra (SaiInfosoft)
Live Demo: https://sdnai.saiinfosoft.co.in

👨‍💻 About the Creator – Shailendra

Founder, SaiInfosoft | Creator of Krishna Software

• Creator of **Krishna Software** – capable of processing **60,000 lines of code per second**.

• Awarded: **"World's Largest Certified Developer"** with **170+ certifications**.

• Top **1%** in more than **60 technologies** and top **5%** in **70+ technologies** on Elance (now Upwork).

• Certification screenshots available at saiinfosoft.co.in/certifications.

⚠️ **Disclaimer**
This document describes the **mathematical foundation** and **high-level architecture** of Sudarshan Nano AI (SDN-AI).
Training details, data generation pipelines, inference engine implementation, and proprietary code are not disclosed.
SDN-AI is **not** a replacement for Large Language Models (LLMs). It is a specialised engine for real-time, deterministic, low-resource automation tasks.

Abstract

All modern neural networks – including Transformers – share a common static primitive: `y = f(Wx + b)`, where weights `W` are fixed after training. This limits adaptability and forces high memory/compute for complex tasks.

SDN-AI replaces this with a **system of three coupled equations** that generate **input-dependent weights** using a **ternary static seed**, a **tiny router**, and a **codebook**. The entire model fits in **<3 MB**, runs on **CPU only**, and achieves **sub-millisecond inference** with **KB-scale runtime memory**. We present the mathematical formulation, compare it with Transformers, and provide live benchmarks.

1. The Limitation of Static Networks

Whether a CNN, RNN, or Transformer, every neural network is built upon:

```
y = f(Wx + b)
```

- `W` and `b` are learned once and remain **fixed** after training.
- The architecture may vary (spatial, temporal, attention), but **the weights never adapt to the input**.

This static nature is the root cause of inefficiency for edge deployment: large models are required for complex tasks, yet they cannot change their computation per input.

2. SDN-AI: From Static to Dynamic Equations

SDN-AI replaces the single static equation with a **system of three equations** that work together.

2.1 Static Seed Equation (baseline)

We store a **static base** using **ternary weights** $W_s \in \{-1,0,1\}^{m \times d}$:

```
y_base = f(W_s x + b)
```

Ternary weights reduce memory by **16×** (2 bits per weight) and turn multiplications into additions/subtractions. However, the model is still static.

2.2 Adaptation Equation (per-input modulation)

A **modulation vector** $m(x)$ is generated based on the input x. We use a hybrid mechanism:

- **Codebook selection**: A tiny router $R(x)$ outputs a distribution over K pre-learned codes $\{c_1,\dots,c_K\}$, each $c_i \in \mathbb{R}^d$ (or $\

\mathbb{R}^m \)). The selected code is \(c = c_{i^*} \) where \(i^* = \arg\max R(x) \).
- **Residual modulation (optional)**: A low-rank generator \(h(x) \) produces a small ternary delta \(\delta \in \{-1,0,1\}^d \) (or zero to save cost).
- **Effective modulation**: \(m(x) = c + \delta \) (clipped to ternary).

Thus:

```
m(x) = CodebookSelect(R(x)) + δ(x)
```

\(R(x) \) is a tiny neural network (e.g., a few fully connected layers with total parameters < 0.1 MB).

2.3 Dynamic Weight Equation

The static seed is modulated element-wise (with broadcasting) to obtain a dynamic weight matrix:

```
W_dyn(x) = W_s ⊙ (1 + m(x))
```

Because \(m(x) \) is ternary, \(W_{\text{dyn}} \) remains a combination of \(-W_s \), \(0 \), and \(+W_s \). No full-precision floating-point multiplications are needed to compute \(W_{\text{dyn}} \).

2.4 Final Output Equation

The forward pass uses the dynamically generated weight:

```
y = f( W_dyn(x) x + b )
```

3. Comparison with the Standard Static Equation

Aspect	Standard Model	SDN-AI
Weight generation	None; \(W\) fixed.	Per-input: \(W_{\text{dyn}}(x) = W_s \odot (1 + m(x)) \)
Adaptability	None.	Yes – each input activates a different subset of the static seed.
Equation count	One.	Three coupled equations (static seed, adaptation, forward).
Inference cost	One matmul \(y = f(Wx + b) \)	One matmul + cheap router + (optional) residual generator.

Aspect	Standard Model	SDN-AI
Memory footprint	Large (32-bit floats).	Tiny: ternary weights + small router + codebook (< 3 MB total).
Arithmetic type	Floating-point.	Integer (ternary: add/sub only).
Determinism	100%	100% (router argmax deterministic).

4. How SDN-AI Differs from Transformers (Equation-Level)

A Transformer uses **attention**:

```
Attention(Q,K,V) = softmax(QK^T/√d_k) V
```

with $Q = xW_Q,\; K = xW_K,\; V = xW_V$. All weight matrices W_Q,W_K,W_V are **static**. Attention computes a weighted average of values based on input similarity, but **the weights that produce Q, K, V are fixed**. Adaptation is limited to the input content, not to the internal weight structure.

SDN-AI, on the other hand, **modifies the weights themselves** per input. Instead of only re-weighting existing features, it can dynamically select which neurons are active – akin to having a set of “expert” weight configurations and a router that picks one per input.

Mathematically:

- **Transformer**: $y = \text{softmax}\bigl((xW_Q)(xW_K)^T\bigr)(xW_V)$ – all weights static.
- **SDN-AI**: $y = f\bigl((W_s \odot (1+m(x))) x + b\bigr)$ – weights depend on x via $m(x)$.

5. Constraint Embedded in the Equation

Instead of enforcing size constraints via loss penalties (e.g., L1), SDN-AI **embeds the constraint directly into the equation structure**:

- **Ternary representation**: forces memory to $\lceil \log_2 3 \rceil = 2$ bits per weight.
- **Codebook size**: K is chosen so that total parameters (static seed + codebook + router) ≤ 3 MB.
- **Low-rank residual**: if used, factorised to keep parameter count negligible.

Thus, the constraint is **not an after-training optimisation** – it is part of the mathematical definition of the model.

6. Live Benchmarks (from the public demo)

All metrics are taken from the live demo running on a standard Ubuntu server (no GPU, 16 GB RAM, Ryzen 7 2700X). The model used is `ticket_classifier_v7` (3-class classification of support tickets).

Metric	Value
Inference latency (mean)	0.48 ms
Peak memory per request	63.52 KB
Model size (ONNX)	2.0 MB
Fixed embedding model (Model2Vec)	30 MB (loaded once, shared)
Concurrency	100+ users without memory growth

These numbers are independently verifiable at https://sdnai.saiinfosoft.co.in. The demo shows **sub-millisecond inference** and **KB-scale runtime memory** for a real neural classifier.

7. Use Cases (Where SDN-AI Excels)

SDN-AI is designed for **ultra-efficient, deterministic, domain-specific automation**. Example applications:

- **Voice-controlled industrial machines** – real-time commands on a solar-powered drone.
- **Offline medical triage** – prioritise patients in remote clinics with no internet.
- **Predictive maintenance** – run on $10 microcontrollers to prevent downtime.
- **Real-time quality control** – classify defects on a production line.
- **Carpet / product designer** – convert natural language into customised outputs (SVG, recommendations).
- **Email / ticket auto-categorisation** – instant, private, zero-cost.

8. Conclusion

SDN-AI redefines the fundamental equation of neural networks from:

```
y = f(Wx + b)   (static)
```

to:

```
m(x) = CodebookSelect(R(x)) + δ(x)      (adaptation)
W_dyn(x) = W_s ⊙ (1 + m(x))            (dynamic weight)
y = f( W_dyn(x) x + b )                (forward)
```

where \(W_s \) is ternary, \(R \) is tiny, and the entire system fits in **<3 MB**. This shift enables **per-input adaptability**, **extreme efficiency**, and **deployment on resource-constrained devices** – a fundamental departure from the Transformer paradigm.

SDN-AI is not an LLM replacement. It is a different tool for a different class of problems: real-time, low-power, deterministic automation. We invite researchers and engineers to explore the mathematics and verify the live demo.

9. Disclaimer & IP Notice

- The equations and high-level architecture described in this document are **open for discussion and citation**.
- **Training algorithms, data generation pipelines, and inference engine implementation remain proprietary.**
- This document does not constitute a license to use, copy, or reverse-engineer any part of the SDN-AI system.
- For licensing or collaboration inquiries, please contact the author via GitHub.

Shailendra | SaiInfosoft

Live Demo
GitHub

YouTube LinkedIn X